an extensions validator available, but it validates against strings, not file uploads.

When the file is uploaded, you will rename it to the city's name. That way, you can ensure each city's image is uniquely named, and you don't accidentally overwrite another city's image should there be a naming clash. It also mitigates against overwriting a critical file on the server, should the uploaded file contain a malicious filename. Once the upload has been processed successfully, you will use the PRG (post-redirect-get) pattern to redirect to another page where the image will be displayed, having used `TempData` to store the city name and the image name.

Let's start with the custom validator. Add a C# class file to the existing ValidationAttributes folder, and name it UploadFileExtensionsAttribute.cs. Following is the full code for the validator, including the using directives.

Listing 6.24 The `UploadFileExtensions` custom validator

```
using System.ComponentModel.DataAnnotations;

namespace CityBreaks.ValidationAttributes
{
    public class UploadFileExtensionsAttribute : ValidationAttribute
    {
        private IEnumerable<string> allowedExtensions;                ❶
        public string Extensions { get; set;  }                       ❷
        protected override ValidationResult IsValid
        ➥ (object value, ValidationContext validationContext)
        {
            allowedExtensions = Extensions?                           ❸
                .Split(new char[] { ',' },                            ❸
                       ➥ StringSplitOptions.RemoveEmptyEntries)      ❸
                .Select(x => x.ToLowerInvariant());                   ❸
            if (value is IFormFile file &&
                ➥ allowedExtensions.Any())
            {
                var extension =                                       ❹
                ➥ Path.GetExtension(file.FileName.ToLowerInvariant());❹
                if (!allowedExtensions.Contains(extension))           ❹❺
```